EU AI Act Ontology Documentation

1. Scope

This ontology provides a structured representation of Regulation (EU) 2024/1689 - the EU Artificial Intelligence Act. Rather than modelling every article, it focuses on the core regulatory structure governing AI systems in the EU. The ontology captures four main aspects:

AI system categories: AI systems are represented according to the Act's risk-based framework, including prohibited AI practices (Article 5), high-risk systems (Annex III), and general-purpose AI models.

Actors in the regulatory ecosystem: Key actors and authorities are represented, including providers, deployers, importers, distributors, authorised representatives, and supervisory authorities (e.g., AI Office, European AI Board, national competent authorities).

Obligations and requirements: The ontology models major legal obligations imposed on actors, such as conformity assessment, transparency obligations, post-market monitoring, and quality management. Technical requirements applicable to high-risk systems (e.g., risk management, data governance, human oversight, robustness, and documentation) are also included.

Risk classification rules: The ontology reflects how AI systems are classified within the four-tier risk framework defined by the Act.

The full text of the regulation (113 articles and annexes) was used as input to the extraction pipeline. However, the ontology is intentionally selective and focuses on the main regulatory entities and relationships rather than modelling the Act article-by-article.

2. AI-Assisted Pipeline

The ontology was constructed using an AI-assisted pipeline based on **Microsoft GraphRAG (v3.0.5)** with Gemini3 as the LLM.

Stage	Description
1. Data extraction	***01_scrape_act.py***- Scraped the EU AI Act text from EUR-Lex into a single text file
2. Knowledge graph extraction	***02_run_graphrag.py***- Ran GraphRAG indexing with a domain-tailored entity extraction prompt. The text was chunked (800 tokens with 150 overlap) and processed using gemini-3.1-flash-lite-preview for entity and relationship extraction and gemini-embedding-001 for embeddings. ***02-2_inspect_graph_output.py*** - Exported the GraphRAG parquet files to human-readable files.
3. Ontology construction	***03_graph_to_owl.py*** **-** Combined extracted entities from GraphRAG with a manually designed class hierarchy
4. Validation & Evaluation	***Loaded Protégé 5.6-*** the OWL ontology and ran the HermiT reasoner to verify logical consistency. ***04_validate.py , 05_evaluate.py*** - Performed SHACL shape validation and calculated ontology metrics.

3. Key Modelling Decisions

GraphRAG : Direct LLM prompting struggles with long legal texts because it is limited by context-window size and produces inconsistent, non-deduplicated output across calls. GraphRAG solves this by chunking the full text, extracting entities and relationships per chunk with a structured prompt, and then merging duplicates across chunks into a single, deduplicated knowledge graph, yielding more systematic coverage than any single prompt could achieve. Additionally, GraphRAG uses an embedding model (we choose `gemini-embedding-001`) to compute vector representations of each text chunk, entity description, and community report; these embeddings enable semantic similarity search at query time, allowing the system to retrieve contextually relevant subgraphs even when the query wording differs from the source text.

The GraphRAG pipeline processed the text in 800-token chunks with 150-token overlap, producing 1,444 entities and 2,473 relationships across 15 entity types (defined in the prompt). To improve extraction quality, three safeguards were used:

- Domain-specific extraction prompt defining 15 main entity types relevant to the EU AI Act.
- Manually curated ontology backbone consisting of core classes and relationships.
- SHACL validation to detect missing labels and incomplete entities.

The ontology loads successfully in Protégé and is logically consistent.

Class vs. individual: Broad legal concepts (e.g., `AISystem`) are modelled as OWL classes. Specific instances from the Act (e.g., `BiometricIdentificationSystem`) are named individuals typed to those classes.

Hierarchical structure : GraphRAG extracts flat entity types, but the ontology introduces a hierarchy. For example: Provider ⊑ Actor , SystemicRiskGPAI ⊑ GeneralPurposeAIModel ⊑ AISystem

Relationship detection: GraphRAG outputs relationship descriptions in natural language. These were mapped to predefined ontology properties (e.g., hasObligation, enforces). Unmatched relationships default to relatedTo.

Namespace reuse: `skos:definition` for concept definitions, `dcterms:description` and `dcterms:source` for ontology metadata, `rdfs:label` for display names.

4. Competency Questions and SPARQL Queries

No.	Competency Question	SPARQL Query
CQ1	What AI practices are prohibited under the EU AI Act?	`SELECT ?s ?l WHERE { ?s rdf:type euai:ProhibitedAIPractice ; rdfs:label ?l }`
CQ2	What obligations does a Provider have?	`SELECT ?p ?pl ?o ?ol WHERE { ?p rdf:type euai:Actor ; rdfs:label ?pl ; euai:hasObligation ?o . ?o rdfs:label ?ol . FILTER(regex(?pl, "provider", "i")) }`
CQ3	Which AI systems are classified as high-risk?	`SELECT ?s ?l WHERE { ?s rdf:type euai:HighRiskAISystem ; rdfs:label ?l }`
CQ4	Which authorities enforce which obligations?	`SELECT ?a ?al ?o ?ol WHERE { ?a rdf:type euai:Authority ; euai:enforces ?o ; rdfs:label ?al . ?o rdfs:label ?ol }`
CQ5	What technical requirements apply to high-risk AI systems?	`SELECT ?r ?l WHERE { ?s rdf:type euai:AISystem ; rdfs:label ?sl ; euai:relatedTo ?r . ?r rdf:type euai:Requirement ; rdfs:label ?l . FILTER(regex(?sl, "high.risk", "i")) }`

5. Validation and Evaluation

The ontology was validated using SHACL constraints and automated reasoning.

Preliminary evaluation was performed using:

- Structural metrics (class depth, property/class ratio, annotation coverage)
- Competency question testing using SPARQL queries

7. Artefacts -

GIT Link : https://github.com/ArchSid/EU_AI_Act_Ontology

Artefact	Path
Ontology (OWL)	ontology/eu-ai-act.owl
Ontology (Turtle)	ontology/eu-ai-act.ttl
Entity Extraction Prompt - RAG	graphrag/prompts/ 2-entity_extraction.txt
GraphRAG Configuration	graphrag/settings.yml
Pipeline Scripts	scripts/01_scrape_act.py, scripts/02_run_graphrag.py, scripts/03_graph_to_owl.py, scripts/04_validate.py, scripts/05_evaluate.py
Evaluation Report	ontology/evaluation_report.txt
Structural Metrics README	docs/evaluation_structural_metrics.md
SPARQL Queries	sparql/sparql_competency_queries.sparql
Validation Report	ontology/validation_report.txt
Validation README	docs/validation_results.md
Input Data	data/input/eu_ai_act.txt